



Lecture 15: Side Effects

INTERVIEW QUESTION

[> CORE](#)

How do you run a side effect when something reactive changes, and how do you clean it up?

You need to save a value to localStorage every time it changes, and remove an event listener when the effect re-runs or the component dies. Where does that logic live?

While `$derived` is used to calculate new data for your UI, sometimes you need to step outside of the UI and interact with the external world. These external interactions are called side effects.

The Problem with Unmanaged Effects

Imagine you are building a document editor that automatically saves the user's progress to the browser's `localStorage` every time they type. If you just put

```
localStorage.setItem('doc', text)
```

floating loose in your `<script>` block, it will only run exactly once when the component is first created. When the user types, the `text` variable updates, but the loose code never runs again.

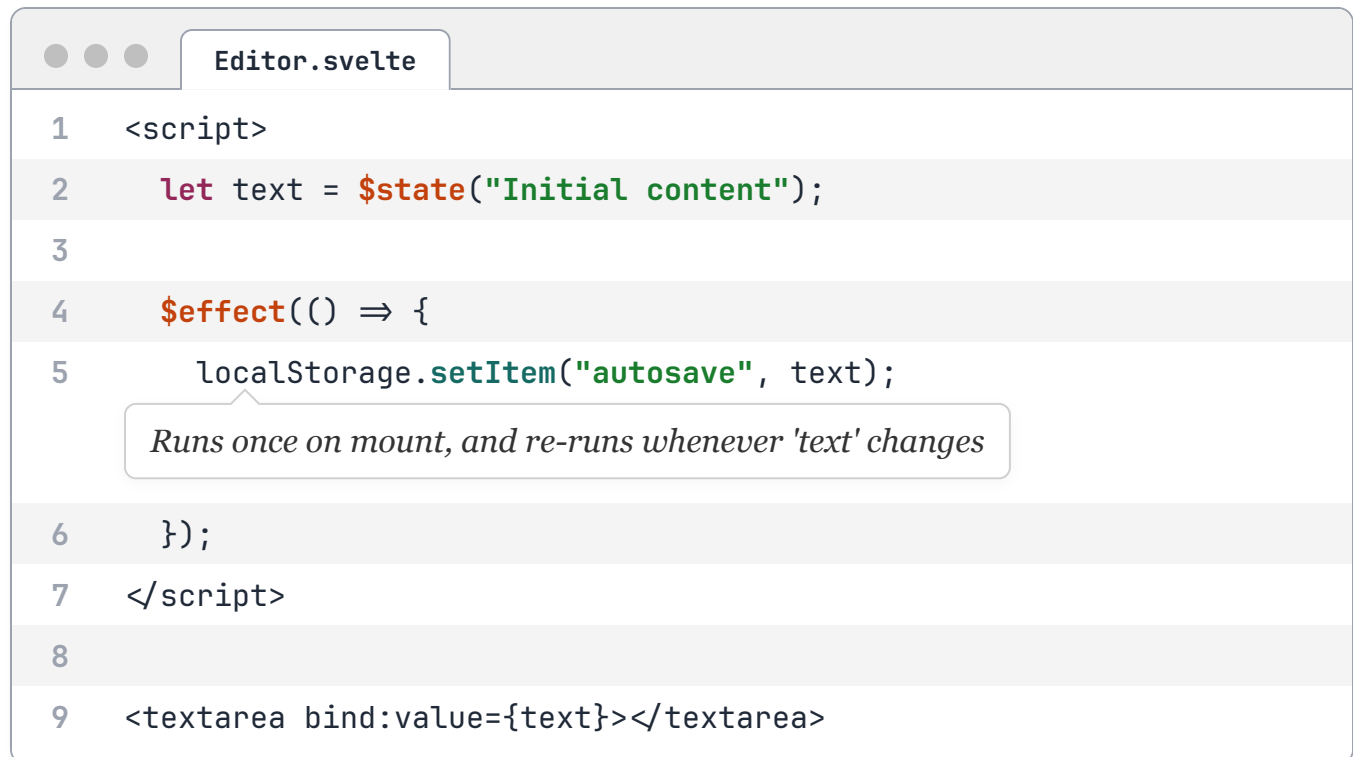
Conversely, if you manually add a

```
window.addEventListener('resize', updateLayout)
```

inside a component but forget to remove it when the user navigates away, you have created a memory leak. Every time the user visits that page, another permanent event listener is added, eventually grinding the browser to a halt. You need a safe, reactive wrapper for external code.

The `$effect` Rune

To solve this, Svelte 5 provides the `$effect` rune. It runs a block of code after the component has mounted to the DOM, and automatically re-runs that code whenever any reactive state inside it changes.



```
1  <script>
2    let text = $state("Initial content");
3
4    $effect(() => {
5      localStorage.setItem("autosave", text);
6    });
7  </script>
8
9  <textarea bind:value={text}></textarea>
```

Runs once on mount, and re-runs whenever 'text' changes

When you use the `$effect` rune, you do not need to explicitly declare your dependencies. Svelte's compiler automatically tracks that `text` was read inside the function, and wires up the reactivity under the hood.

KEY TAKEAWAY

Never use `$effect` to synchronize internal state (e.g., updating variable `B` when variable `A` changes). That is what `$derived` is for. Only use `$effect` when you are explicitly stepping outside the Svelte reactivity system, like writing to the DOM, calling an API, or using `localStorage`.

Cleanup Functions

Many side effects create ongoing processes that need to be dismantled, such as setting intervals or attaching event listeners to the `window`.

To clean these up, your `$effect` function simply returns another function. This returned function is the **cleanup phase**. Svelte will automatically execute this cleanup function right before the effect runs again, and right before the component is destroyed.



```
1  <script>
2    let count = $state(0);
3
4    $effect(() => {
5      const interval = setInterval(() => {
6        count++;
7      }, 1000);
8
9      return () => {
10         clearInterval(interval);
11         };
12     });
13  </script>
```

Cleans up the previous interval to prevent memory leaks

Without the cleanup function, the component would spawn a brand new permanent interval timer every time the effect re-ran, immediately crashing the app.

Summary

When you need to interact with external systems outside of Svelte's reactivity model, you use the `$effect` rune. It runs its code block automatically on mount and whenever its internal reactive dependencies change. To prevent memory leaks when managing ongoing processes like timers or event listeners, the effect should return a cleanup function, which Svelte automatically executes before the next run and upon component destruction.

	<code>\$derived</code>	<code>\$effect</code>
Purpose	Calculating data for the UI	Interacting with the external world
Timing	Runs before the DOM updates	Runs after the DOM updates
Cleanup	Not applicable	Returns a cleanup function